SMALL-CAP GROWTH TRACKER · DOC 22

저장소 구조

Repository Structure & Collaboration Workflow

항목	내용
프로젝트	소형주 전문 수주잔고 & CAPEX 추적기
팀원	20225085 김수린 · 20232565 정수연
문서 버전	v0.1 (초안)
작성일	2026. 09. 21

1. 문서 개요

이 문서는 팀 저장소의 구성과 운영 규칙을 정의합니다. 저장소는 외부 서비스가 아닌 자체 Git 서버인 GitSalt에 호스팅합니다. 코드와 문서를 어디에 두는지, 두 팀원이 어떤 절차로 작업을 합치는지, 무엇을 저장소에 올리면 안 되는지를 다룹니다. 시스템 구조 자체는 '04-아키텍처(초안)' 문서를 참조합니다.

> **저장소(Repository)란**
> 프로젝트의 모든 파일과 그 **변경 이력**을 함께 담는 공간입니다. 일반 폴더에는 현재 상태만 남지만, 저장소에는 누가, 언제, 무엇을, 왜 바꿨는지가 커밋(commit) 단위로 기록되어 과거 어느 시점으로든 되돌릴 수 있습니다.
> 각 팀원의 컴퓨터에 있는 **로컬 저장소**에서 작업하고, GitSalt 서버에 있는 팀 공용 **원격 저장소**로 푸시(push)해 합칩니다.

2. 저장소 기본 정보

항목	내용
저장소 이름	smallcap-tracker
주소	https://gitsalt.com/ksurin2003/smallcap-tracker.git
공개 범위	비공개 — 팀원과 담당 교수·조교만 접근 (API 키 유출 방지)
협업자	김수린, 정수연 (개발자 권한), 교수님·조교 (읽기 권한, 필요 시)
기본 브랜치	main
주 언어	Python 3.11+

3. 디렉터리 구조

폴더 구조는 아키텍처 문서의 구성 요소와 1:1로 대응하도록 설계했습니다. 폴더 이름만 보고도 해당 코드가 시스템의 어느 부분인지 알 수 있게 하는 것이 목표입니다.

```
smallcap-tracker/
├── .ci/
│   ├── ci.yml                    # 자동 테스트 설정 (서버 CI 지원 시)
│   └── review_checklist.md       # 병합 요청 리뷰 체크리스트
├── tracker/                      # 핵심 파이썬 패키지
│   ├── domain/                   # 투자 신호 규칙 (순수 함수)
│   │   ├── signals.py            #   수주잔고 비율, 턴어라운드 태그
│   │   ├── matrix.py             #   듀얼 매트릭스 4분면 라벨링
│   │   └── risk_filters.py       #   FR-07·08 리스크 필터
│   ├── pipeline/                 # 배치 워커 4단계
│   │   ├── ingest.py             #   ① 수집
│   │   ├── extract.py            #   ② 추출·검증
│   │   ├── compute.py            #   ③ 신호 계산
│   │   ├── brief.py              #   ④ AI 요약
│   │   └── run_batch.py          #   전체 실행 진입점
│   ├── clients/                  # 외부 API 호출
│   │   ├── dart_client.py
│   │   ├── price_client.py
│   │   └── llm_client.py
│   ├── db/                       # DB 연결·스키마·조회
│   │   ├── schema.sql
│   │   ├── connection.py
│   │   └── repositories.py
│   └── config.py                 # 환경변수 읽기
├── app/                          # Streamlit 웹 화면
│   ├── Home.py
│   └── pages/
│       ├── 1_수주잔고_랭킹.py
│       ├── 2_CAPEX_캘린더.py
│       └── 3_듀얼_매트릭스.py
├── prompts/                      # LLM 프롬프트 (버전별 보관)
│   └── extract_backlog_v1.txt
├── scripts/                      # 수동 실행 도구
│   └── reparse.py                #   저장된 원문 재파싱
├── tests/
│   ├── domain/                   #   순수 함수 단위 테스트
│   └── fixtures/                 #   테스트용 샘플 공시
├── docs/                         # 제출 문서, 설계 문서
├── .env.example                  # 환경변수 목록 (값 없음)
├── .gitignore                    # Git에서 제외할 파일
├── requirements.txt              # 파이썬 패키지 목록
└── README.md                     # 프로젝트 소개, 실행 방법
```

3.1 폴더별 역할과 아키텍처 대응

폴더	역할	아키텍처 대응	주 담당
tracker/domain	투자 신호 판단 규칙. DB·API에 의존하지 않는 순수 함수만 둠	도메인 로직 (7장)	공동
tracker/pipeline	배치 4단계. 각 파일이 한 단계를 담당	배치 워커 (4장)	데이터 담당
tracker/clients	DART, 시세, LLM 등 외부 호출을 한곳에 모음	외부 시스템 연동	데이터 담당
tracker/db	스키마 정의와 SQL 조회 함수	PostgreSQL (5장)	공동
app	Streamlit 화면. mart 테이블만 조회	웹 앱 (6장)	서비스 담당
prompts	LLM 프롬프트를 코드와 분리해 버전별 보관	추출·요약 단계	데이터 담당
scripts	재파싱 등 수동 실행 도구	재파싱 (4.1절)	데이터 담당
tests	단위 테스트와 샘플 데이터	전체	공동
docs	계획서, 아키텍처, 저장소 구조 등 제출 문서	-	공동

3.2 구조 설계 시 고려한 점

- **외부 호출 격리**: DART·LLM 호출 코드를 `clients` 에 모아 두면, API 사용법이 바뀌거나 LLM 모델을 교체할 때 이 폴더만 수정하면 됩니다. 테스트할 때는 가짜(mock) 응답으로 쉽게 대체할 수 있습니다.
- **프롬프트 파일 분리**: 프롬프트를 코드 안에 문자열로 넣지 않고 `prompts` 폴더에 v1, v2 처럼 버전을 붙여 보관합니다. fact 테이블에 어떤 프롬프트 버전으로 추출했는지 기록하면 재파싱 전후 결과를 비교할 수 있습니다.
- **화면과 로직 분리**: `app` 폴더는 화면 표시만 담당하고 계산은 하지 않습니다. 추후 FastAPI·React 로 확장할 때 `api`, `web` 폴더를 추가하기만 하면 되고 `tracker` 패키지는 그대로 재사용합니다.

4. 주요 설정 파일

4.1 .gitignore

Git이 추적하지 않을 파일 목록입니다. 비밀 정보, 자동 생성 파일, 개인 환경 파일을 제외합니다.

```
# 비밀 정보 – 절대 커밋 금지
.env


# 파이썬 자동 생성 파일
__pycache__/
*.pyc
.venv/


# 로컬 데이터·로그
data/
*.log


# 에디터·OS
.vscode/
.idea/
.DS_Store
```

4.2 .env.example

실제 키 값은 각자의 `.env` 파일에만 저장하고, 저장소에는 **변수 이름만 적힌** `.env.example`을 올립니다. 새 팀원은 이 파일을 복사해 `.env`를 만들고 자신의 키를 채웁니다.

```
DART_API_KEY=
LLM_API_KEY=
DATABASE_URL=postgresql://user:password@host:5432/tracker
BATCH_TIME=18:30
```

4.3 README.md

저장소 첫 화면에 표시되는 문서입니다. 프로젝트 한 줄 소개, 설치 방법, 배치 실행 방법, 웹 앱 실행 방법, 폴더 구조 요약, 팀원 정보를 담습니다.

5. 브랜치 전략

2인 팀에 맞게 **Git Flow를 단순화한 방식**을 사용합니다. 브랜치(branch)는 원본을 건드리지 않고 독립적으로 작업하기 위한 '작업용 복사본 줄기'입니다.

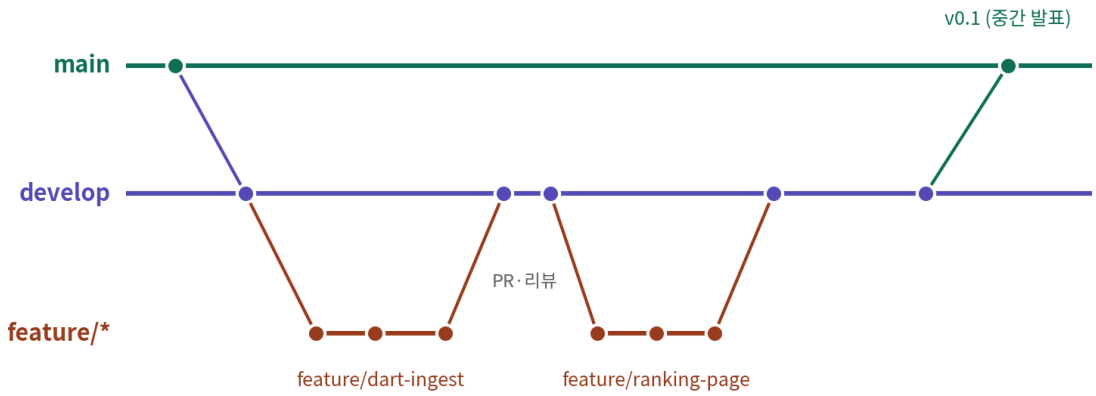


그림 1. 브랜치 운영 흐름

브랜치	용도	규칙
main	항상 실행 가능한 안정 버전. 발표·제출 시점의 코드	직접 커밋 금지. develop에서 PR로만 병합. 병합 시 태그(v0.1 등) 부여
develop	개발 중인 기능을 모으는 통합 브랜치	직접 커밋 금지. feature 브랜치에서 PR로만 병합
feature/기능명	기능 하나를 개발하는 작업 브랜치	develop에서 생성, 작업 완료 후 PR. 병합 후 삭제
fix/내용	버그 수정	feature와 동일한 절차
docs/내용	문서 작성·수정	feature와 동일한 절차

브랜치 이름 예시: `feature/dart-ingest`, `feature/backlog-extract`, `feature/ranking-page`, `fix/unit-conversion`, `docs/architecture-draft`

5.1 브랜치 보호 설정

GitSalt 저장소 설정의 브랜치 보호(Protected Branches) 메뉴에서 `main`과 `develop`에 보호 규칙을 설정합니다.

- 병합 요청(PR)을 거치지 않은 직접 푸시 금지
- 병합 전 상대 팀원 승인(Approve) 1 건 필수
- 자동 테스트(CI) 통과 필수

6. 협업 워크플로우

기능 하나를 개발해서 합치기까지의 전체 절차입니다. GitSalt 서버가 병합 요청이나 이슈 기능을 제공하지 않는 경우, 이슈는 별도 작업 목록으로, 리뷰는 상대 팀원이 develop 병합 전에 코드를 확인하는 절차로 대체합니다.

1. **이슈 등록**: GitSalt 이슈 트래커에 할 일을 등록합니다. 예: '#12 DART 정기보고서 수집 기능'
2. **브랜치 생성**: develop 에서 작업 브랜치를 만듭니다.
3. **작업·커밋**: 의미 있는 단위로 나눠 자주 커밋합니다.
4. **푸시**: 작업 브랜치를 GitSalt 서버에 올립니다.
5. **병합 요청(PR) 생성**: develop 으로 병합을 요청하고, 본문에 관련 이슈 번호(Closes #12)와 변경 내용을 적습니다.
6. **코드 리뷰**: 상대 팀원이 코드를 확인하고 의견을 남기거나 승인합니다.
7. **병합**: 승인과 테스트 통과 후 병합하고, 작업 브랜치는 삭제합니다.

```
git switch develop
git pull origin develop                 # 최신 상태로 갱신
git switch -c feature/dart-ingest       # 작업 브랜치 생성

# ... 코드 작성 ...
git add tracker/pipeline/ingest.py
git commit -m "feat: DART 정기보고서 수집 기능 추가"
git push -u origin feature/dart-ingest  # 이후 GitSalt에서 PR 생성
```

> **충돌(Conflict) 줄이는 요령**
> 작업 시작 전에 항상 `git pull`로 최신 코드를 받습니다. 브랜치는 며칠 이상 오래 끌지 말고 작게 나눠 자주 병합합니다. 폴더별 주 담당을 정해 두면 같은 파일을 동시에 고치는 일이 줄어듭니다.

7. 커밋 메시지 규칙

형식: 타입: 변경 내용 요약 (한국어, 50자 이내)

타입	의미	예시
feat	새로운 기능 추가	feat: 수주잔고/시총 비율 계산 함수 추가
fix	버그 수정	fix: 백만원 단위 변환 오류 수정
docs	문서 작성·수정	docs: 아키텍처 초안 추가

타입	의미	예시
refactor	기능 변화 없는 코드 개선	refactor: DART 호출 코드를 clients로 분리
test	테스트 추가·수정	test: 턴어라운드 태그 경계값 테스트 추가
chore	설정, 패키지 등 기타	chore: requirements.txt에 pykrx 추가

8. 보안 규칙

이 프로젝트는 Open DART API 키, LLM API 키, DB 접속 정보를 사용합니다. 다음 규칙을 반드시 지킵니다.

- API 키와 비밀번호는 `.env` 파일에만 저장하고, `.env` 는 `.gitignore` 에 등록합니다.
- 코드 안에 키를 직접 적지 않고 `config.py` 에서 환경변수로 읽습니다.
- 실수로 키를 커밋한 경우, 커밋을 지우는 것만으로는 부족합니다. 이력에 남아 있으므로 **해당 키를 즉시 재발급(폐기)**합니다.
- CI 에서 키가 필요한 경우 서버의 비밀 변수(Secrets·CI Variables) 기능에 등록하고, 해당 기능이 없으면 키가 필요한 테스트는 CI 대상에서 제외합니다.

9. 초기 커밋 계획

팀 저장소 생성 직후 올릴 초기 프로젝트 구조입니다. 기능 코드는 아직 없고, 폴더 골격과 설정 파일, 문서만 포함합니다.

순서	커밋 메시지	포함 내용
1	chore: 프로젝트 초기 구조 생성	전체 폴더, 빈 __init__.py, .gitignore, .env.example, requirements.txt
2	docs: README 및 제출 문서 추가	README.md, docs/ 아래 계획서·아키텍처·저장소 구조 문서
3	chore: CI 설정과 리뷰 체크리스트 추가	.ci/ci.yml, .ci/review_checklist.md
4	(develop 브랜치 생성)	main에서 develop 생성 후 원격에 푸시, 브랜치 보호 규칙 설정

```
git init
git add .
git commit -m "chore: 프로젝트 초기 구조 생성"
git branch -M main
git remote add origin https://[GitSalt 주소]/[팀 계정]/smallcap-tracker.git
git push -u origin main


git switch -c develop
git push -u origin develop
```

10. 역할 분담 (초안)

폴더 단위로 주 담당을 나눠 충돌을 줄이고, `domain`과 `db`는 공동으로 관리합니다. 담당자는 팀 협의 후 확정합니다.

구분	주 담당 폴더	주요 작업
데이터 담당	tracker/pipeline, tracker/clients, prompts, scripts	DART·시세 수집, Regex·LLM 추출, 배치 스케줄링, 재파싱
서비스 담당	app, tests	Streamlit 화면(랭킹·캘린더·매트릭스), 테스트 작성, README 관리
공동	tracker/domain, tracker/db, docs	신호 규칙 정의, 테이블 설계, 제출 문서 작성, 상호 코드 리뷰